Build Automation Using Maven

Automating the build, dependency management, and packaging of a Java application using Apache Maven.

Overview

This project is part of a DevOps internship focused on learning build automation and CI/CD fundamentals.

This project demonstrates how to automate the build process of a Java application using Apache Maven. Maven simplifies tasks such as dependency management, code compilation, testing, and packaging through a standardized project structure and lifecycle. In this project, a simple Java application is created, external dependencies are managed using the Maven pom.xml file, and the application is packaged into an executable JAR file. This automation helps ensure consistent builds and simplifies the process of preparing applications for deployment.

Objectives

- Understand the basic workflow of build automation using Apache Maven.
- Create a standard Maven project using an archetype template.
- Learn how Maven manages dependencies using the pom.xml configuration file.
- Compile and package a Java application using Maven lifecycle commands.
- Generate a deployable JAR artifact from the source code.
- Gain familiarity with Maven plugins and dependency management.

Tools and Technologies Used

- **Java (JDK 17)** – Programming language used to develop the application.
- **Apache Maven** – Build automation and dependency management tool used to compile and package the application.
- **Apache Commons Lang** – External Java library used for string utility functions in the project.
- **Linux / WSL Environment** – Development environment used to run Maven commands and manage the project.

Prerequisites

Before running this project, ensure the following tools are installed on your system:

- **Java JDK 17 or later** – Required to compile and run the Java application.

- **Apache Maven** – Used for build automation, dependency management, and packaging the application.
- **Linux or WSL Environment** – Used to execute Maven commands and manage the project.
- **Internet Connection** – Required for Maven to download project dependencies from the Maven Central Repository.

Verify Installation

Run the following commands to confirm that Java and Maven are installed correctly:

java -version

mvn -version

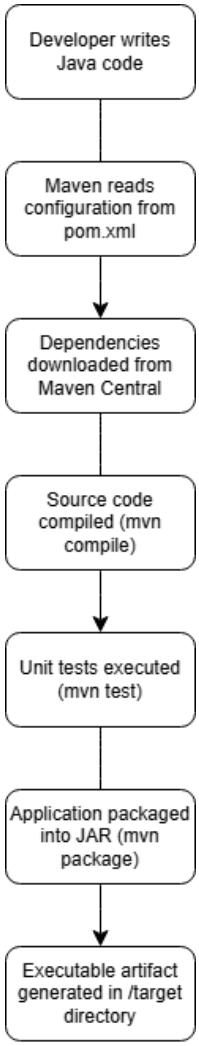
If installed correctly, the commands will display the Java and Maven versions.

Project Architecture / Workflow

The project follows a simple build automation workflow using Apache Maven. The source code is compiled, tested, and packaged into a deployable artifact through Maven's build lifecycle.

Workflow

This automated workflow ensures that the application is built consistently and can be easily deployed or integrated into CI/CD pipelines.



Project Structure

Below is the directory structure of the Maven project.

Directory Explanation

- **pom.xml** – Maven configuration file that defines project dependencies, plugins, and build settings.
- **src/main/java** – Contains the main application source code.
- **src/test/java** – Contains unit test classes used during the testing phase.
- **target** – Output directory where Maven places compiled classes and the packaged JAR artifact after building the project.

Setup and Installation

Follow the steps below to set up and run the project.

1. Install Java

Ensure that Java JDK 17 or later is installed.

```
java -version
```

2. Install Apache Maven

Verify Maven installation using:

```
mvn -version
```

3. Create the Maven Project

Generate the project using a Maven archetype template.

```
mvn archetype:generate \
-DgroupId=com.devops.app \
-DartifactId=maven-demo \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

4. Navigate to the Project Directory

```
cd maven-demo
```

5. Build the Project

Compile and package the application using Maven.

```
mvn clean package
```

After the build completes successfully, the packaged JAR file will be generated inside the target directory.

Implementation Steps

Step 1: Create the Maven Project

A new Maven project was generated using the Maven archetype template. This automatically created the standard Maven project structure including the pom.xml configuration file and source code directories.

```
mvn archetype:generate \
-DgroupId=com.devops.app \
```

```
-DartifactId=maven-demo \

-DarchetypeArtifactId=maven-archetype-quickstart \

-DinteractiveMode=false
```

Step 2: Configure the Maven Project

The pom.xml file was updated to configure the Java version used during compilation.

```
<properties>

  <maven.compiler.source>17</maven.compiler.source>

  <maven.compiler.target>17</maven.compiler.target>

</properties>
```

This ensures the project is compiled using Java 17.

Step 3: Add External Dependency

An external dependency was added to the project to demonstrate Maven's dependency management capabilities.

```
<dependency>

  <groupId>org.apache.commons</groupId>

  <artifactId>commons-lang3</artifactId>

  <version>3.14.0</version>

</dependency>
```

Maven automatically downloads this library from the Maven Central Repository and stores it in the local .m2 repository.

Step 4: Implement Application Logic

The main Java application (App.java) was modified to use the Apache Commons library to trim whitespace from a string and display the processed output.

The program performs the following operations:

- Creates a string with extra whitespace.
- Uses the StringUtils.trim() method from Apache Commons to remove leading and trailing spaces.
- Prints the original string, trimmed result, and string length.

Step 5: Build the Application

The project was compiled, tested, and packaged using the Maven lifecycle command:

mvn clean package

This command performs the following actions:

- Removes previous build files
- Compiles the source code
- Executes tests
- Packages the application into a JAR file

Step 6: Generate Executable JAR

The Maven Shade Plugin was configured to create a **fat JAR** containing the application and all required dependencies. This allows the application to be executed using a single command.

Maven Commands Used

The following Maven commands were used during the development and build process.

Clean

Removes previously generated build files from the target directory to ensure a fresh build.

mvn clean

Compile

Compiles the Java source code and generates .class files inside the target/classes directory.

mvn compile

Test

Runs unit tests located in the src/test/java directory.

mvn test

Package

Packages the compiled code into a deployable artifact such as a JAR file.

mvn package

Clean Package (Combined Command)

This command first removes previous build files and then compiles, tests, and packages the application.

mvn clean package

This command was used to generate the final executable JAR file for the project.

Build Output

After running the build command, Maven generates compiled files and packaged artifacts inside the target directory.

Build Command

mvn clean package

Output Directory

target/

Generated Files

- **Compiled Classes**

target/classes/

Contains the compiled .class files generated from the Java source code.

- **Executable JAR File**

target/maven-demo-1.0-SNAPSHOT.jar

This is the final build artifact generated by Maven.
It contains the application code along with all required dependencies bundled together using the Maven Shade Plugin.

This JAR file can be executed directly using the Java runtime.

Running the Application

After the project has been successfully built, the generated JAR file can be executed using the Java runtime.

Run Command

Navigate to the project directory and run:

java -jar target/maven-demo-1.0-SNAPSHOT.jar

Expected Output

When the application runs successfully, the following output will appear in the terminal:

Original: ' DevOps Maven Project '

Trimmed: 'DevOps Maven Project'

Length: 20

Message is long

Explanation

The program demonstrates the use of an external dependency (Apache Commons Lang) to process a string. It removes extra whitespace using the StringUtils.trim() method and prints the cleaned result along with its length.

Key Learnings

- Understood the role of Apache Maven in automating the build process of Java applications.
- Learned the standard Maven project structure and the purpose of the pom.xml configuration file.
- Practiced using Maven lifecycle commands such as clean, compile, test, and package.
- Gained experience managing external libraries using Maven dependencies.
- Learned how to create an executable fat JAR using the Maven Shade Plugin.
- Understood how Maven downloads dependencies automatically from the Maven Central Repository.

Challenges Faced

Java Version Compatibility Issue

During the build process, Maven failed to compile the project with the following error:

Source option 5 is no longer supported. Use 7 or later.

This occurred because the default configuration in the generated Maven project was set to use an older Java version that is no longer supported by modern JDK versions.

Solution

The issue was resolved by explicitly specifying the Java version in the pom.xml file:

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
```

Missing Dependency at Runtime

When running the packaged JAR file, the application failed with a NoClassDefFoundError related to the Apache Commons library.

This occurred because the default JAR produced by Maven does not include external dependencies.

Solution

The problem was solved by configuring the **Maven Shade Plugin** to generate a fat JAR containing both the application code and all required dependencies.

Conclusion

This project demonstrated how Apache Maven can be used to automate the build process of a Java application. By using Maven's dependency management and lifecycle commands, the application was successfully compiled, tested, and packaged into an executable JAR file. The project also introduced the use of Maven plugins to create a fat JAR that includes all required dependencies. Overall, this exercise provided practical experience with build automation, a fundamental component of modern DevOps workflows.